

Tide

A partially-active AMM with a collateral buffer, shipped as a linch Aqua SwapVM program and a Uniswap v4 hook, governed through ENSv2 and World ID

ETHGlobal Tokyo 2026 · September 26, 2026 · github.com/asengupta07/tide · Sepolia

Abstract. Passive liquidity providers lose money to the first trade of every block: the pool’s price is stale until someone trades, and an arbitrageur who knows the fresh price trades against the whole pool at the old one. Tide shows that arbitrageur only a fraction λ of the maker’s inventory, quotes uninformed follow-on flow against an N -times deeper virtual curve backed by the idle remainder, and bounds how far that virtual curve can be pushed with a drift threshold δ . We prove that steady-state loss-versus-rebalancing falls to $1/(2 - \lambda)$ of a plain constant-product pool (33% saved at $\lambda = 0.5$, 43% at $\lambda = 0.25$) and state the solvency invariant the buffer must satisfy; 10,000-path Monte-Carlo agrees with the closed form to three digits. We also show that the virtual curve is a rebate paid by the LP: with no fee it is drained every block by a plain-then-virtual round trip that needs no price gap, and we prove and enforce on-chain the bound $(N - 1)\delta \leq 2f$ under which a flat fee f makes that round trip, and any stale-anchor play, unprofitable. The same 100-line math library prices both venues: three custom SwapVM opcodes on a redeployed Aqua router and a `beforeSwap` hook with hook-owned reserves, and a cross-venue test asserts identical fills. The parameters live as ENSv2 text records on `eth-usdc.tide.eth`; a manager agent, `manager.tide.eth`, holds an Enhanced Access Control role for exactly λ , N and δ and may write only after the owner completes a fresh World ID authentication. Everything is live on Sepolia and reproducible on a mainnet fork.

1 The problem in one figure

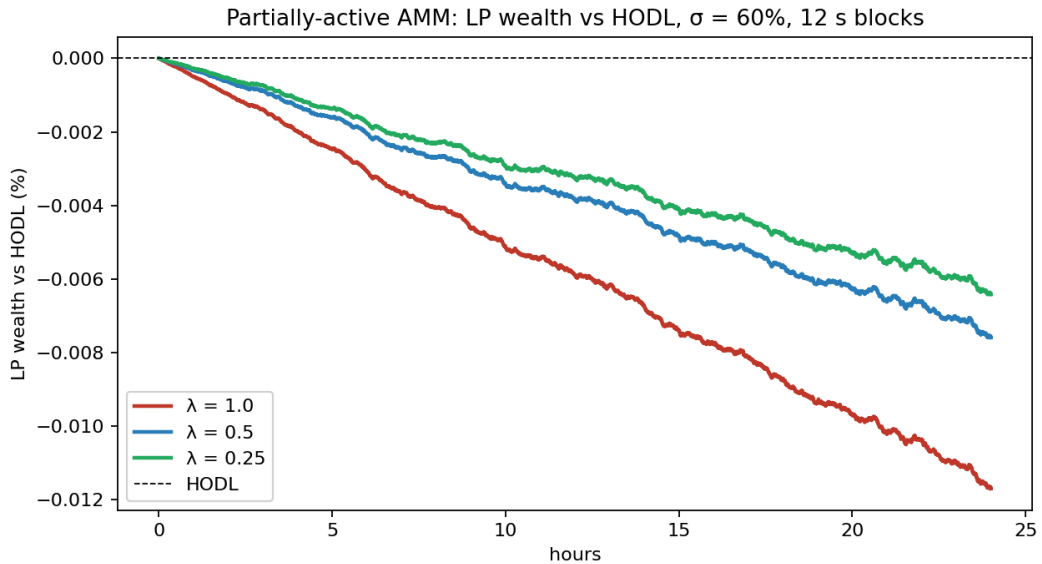


Figure 1: LP wealth relative to HODL over one trading day, $\sigma = 60\%$ annualised, 12-second blocks, 500 paths. A fully active constant-product pool ($\lambda = 1$) bleeds LVR every block; exposing half ($\lambda = 0.5$) or a quarter ($\lambda = 0.25$) of the reserves per block cuts the bleed. Fees are not modelled, so every curve sits below zero and the gaps are pure LVR saved.

Loss-versus-rebalancing (LVR) is the cost of quoting a stale price. Milionis, Moallemi, Roughgarden and Zhang show that for a constant-product pool it accrues at $\sigma^2/8$ of pool value per unit time: at 60% annualised volatility about 4.5% of capital per year before fees. Every shipped mitigation either prices the arbitrage (dynamic and auctioned fees), removes intra-block ordering (batch clearing) or rebates it. None reduces what the arbitrageur can see. Two 2026 papers do, and neither had an implementation: Ko’s partially active AMM [1] and Kim and Park’s collateralized liquidity scaling [2]. Tide is both, in one program.

2 Model and notation

The price P_t follows a geometric Brownian motion with volatility σ ; blocks have length Δ . A constant-product pool with reserves (x, y) has value $E = xP + y$ and marginal price $P = y/x$. Within a block the pool is stale; an arbitrageur trades it to the new price and keeps the difference. For a curve of value V the expected per-block loss is, to first order in Δ ,

$$\text{LVR} = \frac{\sigma^2}{8} \cdot V \cdot \Delta. \quad (1)$$

Tide adds three governed parameters and a fee. $\lambda \in (0, 1]$ is the fraction of reserves exposed per block; $N \geq 1$ is the virtual depth multiplier; δ is the drift bound in basis points; f is a flat fee on the input token, set by the owner, which Proposition 4 ties to N and δ .

3 Mechanism

Per block, per strategy:

1. **Active split (lazy).** On the first quote or fill of a block, `active = lambda * total` for each token; the remainder is passive and invisible to the block. Neither SwapVM nor Uniswap v4 has a block hook, so the split happens inside the first `quote() / swap()` of the block and is persisted only by a fill.
2. **First fill = informed.** The first fill of a block is priced on the plain constant-product curve over the active reserves ($N = 1$). The arbitrageur sees λ of the vault.
3. **Later fills = virtual curve.** Every later fill in the block is priced on $(Nx_a)(Ny_a) = k$ over the current active reserves (x_a, y_a) . A trade of size q sees slippage $\approx q/(Nx_a)$ instead of q/x_a .
4. **Drift bound.** The reserves right after the first fill are the block’s anchor. A later fill that would move the virtual price outside $[P_{\text{anchor}(1-\delta)}, P_{\text{anchor}(1+\delta)}]$ is informed-sized and is re-priced on the active curve.
5. **Solvency and top-up.** Any fill must be deliverable from `active + passive`; passive is read live (the maker’s Aqua balance, or the hook’s ERC-6909 claims). A fill that dips into passive triggers a re-split from the new totals: the buffer tops the active side up at the current marginal price, which creates no arbitrage.
6. **Fee.** Every fill pays a flat fee f on the input token. The curve and the drift guard see the net input; the taker pays gross; the fee stays in the passive buffer until the next re-split. The fee is read from `TideParams` together with λ, N, δ , so the fee a fill pays is by construction the one Proposition 4 bounded δ against.

The SwapVM program is `ACTIVE_SPLIT VIRTUAL_XYC BUFFER_GUARD Salt`. Instruction order is security-critical, so every Tide opcode parses the whole program and reverts unless the three opcodes appear once each in that order (`TideProgram.check`). The v4 hook runs the same six steps inside `beforeSwap` and returns a `BeforeSwapDelta` for the whole amount.

Opcode	Slot	Bank	Encoding
ACTIVE_SPLIT	0x92	balances tuning	[address params]
VIRTUAL_XYC	0x52	swap curves	[address params]
BUFFER_GUARD	0x22	conditions and guards	[address params]

Table 1: Reserved third-party slots claimed in linch’s `OpcodeList.sol`; no linch opcode is overwritten. Each opcode reads λ, N, δ, f from `TideParams`, keyed by the Aqua order hash (or the v4 `PoolId`).

4 Propositions

Proposition 1 (activeness scales LVR). Let a Tide strategy with total equity E expose active reserves $A = \lambda E$ at the top of each block. The block’s first fill faces a constant-product curve of value λE , so its expected loss is λ times Equation 1. In steady state the passive part lags the market and the price gap the arbitrageur closes accumulates; the expected per-block loss converges to

$$\mathbb{E}[\text{LVR}_{\text{Tide}}] = \frac{\sigma^2}{8} \cdot E \cdot \Delta \cdot \frac{1}{2 - \lambda}. \quad (2)$$

Proof. Let $g_t = \ln(P_t/P_t^{\text{pool}})$ be the log gap between the market and the pool’s marginal price at the top of block t , before the fill. The fill closes the gap on the active slice; after the re-split the pool’s price is the reserve-weighted mix of the active part (at P_t) and the passive part (still at the old pool price), so the gap carried into the next block is $(1 - \lambda)g_t$ plus the next return r_{t+1} , which is $N(0, \sigma^2 \Delta)$. Hence g is an AR(1) with coefficient $(1 - \lambda)$ and stationary variance $\sigma^2 \Delta / (1 - (1 - \lambda)^2) = \sigma^2 \Delta / (\lambda(2 - \lambda))$. The constant-product loss for closing a gap g on a curve of value λE is $\lambda E g^2 / 8$ to second order, so $\mathbb{E}[\text{LVR}] = (\lambda E / 8) \cdot \sigma^2 \Delta / (\lambda(2 - \lambda))$, which is Equation 2. The first-order single-block statement is the g -fresh case $g_t = r_t$. $\square$

Simulation confirms the ratio $1/(2 - \lambda)$ to three digits (Table 2). The gap dynamics are exactly those of Section 3 of [1]; we do not re-derive them.

Proposition 2 (the cost, solved numerically). Because only λ of the reserves rebalance, the pool’s token weight w_t drifts from the invariant’s target $\theta = 1/2$. The deviation follows an AR(1) with decay $(1 - \lambda)$ and shock $r_t/4$. The LP chooses λ to minimise

$$\text{LVR}(\lambda) - \text{Fees}(\lambda) + \kappa \sum_t (w_t - \theta)^2, \quad (3)$$

where fees on arbitrage flow are $f \cdot (\lambda E / 4) \cdot \mathbb{E}|r|$ per block. LVR and tracking error scale with σ^2 while fees scale with σ , so the optimum falls as volatility rises. `research/frontier.py` solves Equation 3 on a grid with $f = 1$ bp and κ calibrated so $\lambda^* = 0.5$ at $\sigma = 60\%$; the manager agent reads λ^* off the resulting frontier.

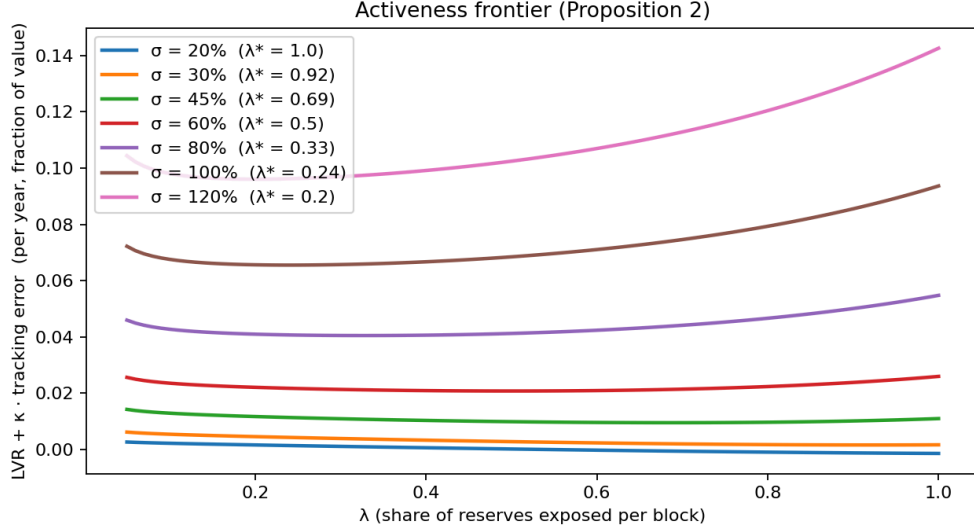


Figure 2: The activeness frontier: objective Equation 3 against λ for several volatilities; λ^* is 1.0 at $\sigma = 20\%$, 0.5 at 60% , 0.24 at 100% .

Proposition 3 (virtual depth with a buffer, solvency invariant). Quoting follow-on flow against $(Nx_a)(Ny_a) = k$ reduces slippage of a trade of size q from $\approx q/x_a$ to $\approx q/(Nx_a)$; at $N = 4$ a trade of 1% of active reserves gets 3.97 times less slippage (Table 3). The virtual curve can promise up to Ny_a , which the pool does not hold. With the drift bound δ measured against the block anchor, the most the virtual curve can be asked to deliver in one block before a re-split is

$$y_{\max} = Ny_a(1 - \sqrt{1 - \delta}), \quad (4)$$

so the strategy is solvent whenever

$$y_a + B \geq Ny_a(1 - \sqrt{1 - \delta}), \quad (5)$$

with B the passive buffer. For the deployed $N = 4$, $\delta = 0.2\%$ the right-hand side is 0.4% of y_a , far inside the active slice; under Proposition 4 the buffer can only engage for extreme settings (a 45% band needs a 67.5% fee), and `BUFFER_GUARD` reverts any fill beyond `active + passive` regardless. `TideMath.maxOutWithinDrift` computes $y_{\max}$ and the test suite checks it is exactly the boundary of `driftExceedsRef`.

Proposition 4 (the virtual curve is a rebate; the fee-rebate bound). A fill on the virtual curve moves the virtual price by some amount and the real active reserves by the same tokens, which on the real curve is N times that price move. After a within- δ fill the real active price sits up to $(N - 1)\delta$ from the virtual one, and the next block's first fill collects the gap. Hence every unit of execution improvement the virtual curve grants is a unit of the LP's inventory at the next re-split: inside the band the improvement over the active curve is at most $(N - 1)\delta/2$ per unit traded.

With $f = 0$ this is extractable at will. Let a taker be first in the block: on the active curve ($N = 1$) she sells x to move the price by $\delta' \geq \delta$, then on the virtual curve buys x back up to the guard, δ from the anchor. Her round trip is a sale on a shallow curve and a purchase on a deep one, and nets, to first order,

$$(N - 1)N \frac{\delta^2}{4} \quad (6)$$

of one side of the active reserves per block, with no price gap and no information; the real reserves are left $(N - 1)\delta$ mispriced for the next arbitrageur on top. Exact simulation with the contract formulas (`research/`, USD 2 M pool, $\lambda = 0.5$, $N = 4$, $\delta = 0.5\%$) gives about USD 28 per block, some 830 times

the LVR that λ saves in that block at $\sigma = 60\%$; at $\delta = 0.05\%$ still 8 times. Neither λ , N nor δ bounds the sum over blocks.

A flat fee f on every input does. The round trip pays f twice and gains at most the rebate; honest one-directional flow pays f once. The rebate can never be farmed if it never exceeds the fee that pays for it:

$$(N - 1)\delta \leq 2f. \quad (7)$$

`TideMath.checkParams` enforces Equation 7 at `init`, at every `set` and at every `setFee`; the venues read f from the same record, so a fill always pays the fee the bound was checked against. Under Equation 7 the numerical search that found the USD 28 loop finds no profitable loop (the unprofitable region extends to $4f/(N - 1)$; the bound keeps a factor two), and the largest admissible one-directional honest flow leaves the LP whole after the following arbitrage. A test performs the round trip on the Aqua venue and asserts the attacker loses and the maker’s inventory is worth no less at the old price.

The bound also settles the ordering question for a stale anchor. A dust first fill anchors the block at the pool’s current price, which arbitrageurs keep within f of the market plus one block of drift, $\sigma\sqrt{\Delta}$ (3.7 bp at $\sigma = 60\%$, 12 s). A follower on the virtual curve gains at most gap $-f - \delta/2$, so once δ exceeds two one-block moves the stale anchor is worth nothing to anyone. The manager therefore keeps δ in the box

$$3\sigma\sqrt{\Delta} \leq \delta \leq \frac{2f}{N - 1}, \quad (8)$$

lowering N when the box is empty ($N = 1$ turns the virtual curve off). At $\sigma = 60\%$, $f = 30$ bp, $N = 4$: $11 \text{ bp} \leq \delta \leq 20 \text{ bp}$; the deployed strategy uses $\delta = 20$ bp. $\square$

5 Simulation

10,000 GBM paths, $\sigma = 60\%$, 12-second blocks, one day, arbitrageur trades the active slice to the new price every block (`research/sim.py`).

λ	LVR / day	simulated ratio	$1/(2 - \lambda)$	saved
1.00	0.01233%	1.000	1.000	0%
0.50	0.00822%	0.667	0.667	33.3%
0.25	0.00705%	0.571	0.571	42.9%

Table 2: LVR per day as a fraction of capital. $\lambda = 1$ reproduces $\sigma^2/8/365 = 0.01233\%$.

N	slippage, trade = 1% of active	vs. $N = 1$
1	0.990%	1.0×
2	0.498%	2.0×
4	0.249%	3.97×
8	0.125%	7.9×

Table 3: Retail slippage on the virtual curve.

6 Implementation

Contract	Role
contracts/src/lib/TideMath.sol	shared math; checked against research/tide_math.py vectors (24 cases)
contracts/src/aqua/instructions/*.sol	the three opcodes and the program-order check
contracts/src/aqua/TideOpcodes.sol , TideRouter.sol	AquaOpcodes + Tide dispatch; router = swap-vm main template, opcode set swapped
contracts/src/aqua/TideApp.sol	program and Aqua order builder; ship() is sent by the maker to the official Aqua registry
contracts/src/TideParams.sol	governed λ, N, δ plus the fee; owner or manager, fee owner-only; Equation 7 enforced at init, set, setFee
contracts/src/v4/TideHook.sol	BaseCustomCurve hook, hook-owned ERC-6909 reserves, pro-rata shares, JIT guard

Table 4: Contract map.

Path	gas	Δ vs plain
plain XYCSwap fill on the same router (taker callback + Aqua push/pull included)	83,226	–
Tide fill, first of block (re-split, fee, storage write)	118,821	+35,595
Tide fill, later in block (virtual curve + guard + fee)	119,551	+36,325
Tide v4 hook swap via PoolSwapTest	~168,000	n/a

Table 5: Gas (Foundry, test/BaselineGas.t.sol). The Tide overhead is three TideParams.get calls, one Aqua balance read in the guard and the program scan; it could be halved by caching parameters per block.

Tests: 46 (forge test): vector parity with Python, every opcode, two swaps in one block, next-block re-split, informed-sized follow-on re-pricing, exact-out beyond inventory reverts, buffer top-up re-split, fee netting and gross-up round trip, quote/swap consistency in both directions and modes, reordered/missing/duplicate program reverts, governance (manager can set, stranger cannot, revoke blocks writes, new λ applies at the next re-split, Equation 7 rejects a δ the fee cannot back, fee is owner-only), the plain-then-virtual round trip loses under the bound, hook JIT guards, and cross-venue parity.

Live on Sepolia: router 0xfDD5...0957, params 0x4608...44FF, hook 0xEcbF...AA88, strategy hash 0xd09a...e4f8; Aqua fill 0xcaea...7773 (0.02 WETH $\rightarrow$ 55.40 USDC, 30 bp fee kept by the maker), hook swap 0x8a06...7497 (quote equals fill). A mainnet-fork script fills against the official Aqua registry with real WETH/USDC.

7 Governance: the parameters are ENS records, the human is World ID

λ, N, δ and the fee are text records on eth-usdc.tide.eth (ENSv2, Sepolia), served by a PermissionedResolver proxy that the strategy owns. manager.tide.eth is an ENSIP-26 agent name whose wallet holds ROLE_SET_TEXT on exactly three EAC resources, keccak(lambda), keccak(N), keccak(delta), granted with grantSetterRoles; on-chain it is the manager of TideParams. It cannot set strategyHash, an address, the resolver, or unregister the name; a script demonstrates each revert. Revocation is one revokeRoles per key plus setManager(0).

The agent proposes λ^* from the frontier and δ from the box of Proposition 4 (three one-block moves at the realised volatility, capped by the fee bound; the fee itself is owner-only). The backend then starts an

OpenID Connect step-up with the World ID for Agents dev environment: `prompt=login, max_age=0`. The ID token is validated server-side (JWKS/RS256, `iss`, `aud`, `nonce`, `iat` ≤ 120 s, `auth_time` not before the request) and its pairwise `sub` must equal the owner bound at setup. Only then does the agent call `setText` and `TideParams.set`. Denied, cancelled, timed-out, replayed and forged responses all leave the records unchanged; a harness drives each path and asserts it.

8 Sponsor integration rationale

linch Aqua. The passive reserve *is* the maker’s wallet balance: Aqua pulls only at fill time and the Shared Liquidity Ratio lets one balance back several positions. Tide needed a place where “the pool does not hold what the virtual curve promises” is the normal state, and Aqua is that place. SwapVM’s byte programs let the three mechanisms be composed as opcodes in reserved slots on the official router template.

Uniswap v4. The hook proves the mechanism is venue-agnostic: same library, identical fills. `BeforeSwapDelta` carries the custom curve; hook-owned ERC-6909 claims are the buffer. The missing block boundary callback is the main friction (see `FEEDBACK.md`).

ENSv2. Parameters must be readable by anyone, writable by exactly one scoped role, and auditable. Per-record EAC roles on a Permissioned Resolver and agent-as-namespace are the only primitive that gives per-key delegation with a one-call revoke.

World ID for Agents. Changing λ moves the owner’s money; a replayed session must not be enough. Fresh authentication with a pairwise subject is the minimum assurance that *this* human approved *this* change now.

9 Limitations and future work

- The “first fill of the block is the informed one” model is the PA-AMM assumption; a retail order that happens to be first pays the shallow curve. Proposition 4 removes the profit from taking that slot on purpose, not the cost of taking it by accident.
- Steady-state LVR saving is capped at 50% as $\lambda \rightarrow 0$; the drift cost grows correspondingly. The frontier makes the trade-off explicit, but its fee term is first order and solved at $f = 1$ bp; at the deployed 30 bp the fee changes *when* arbitrage happens (the gap must exceed f first), which the LVR model does not capture. λ^* is the LVR-versus-tracking optimum, not a fee-calibrated one.
- The δ rule (three one-block moves) has a proven safe side and a heuristic target; the optimal band for a given flow mix is open.
- Parameters are mirrored on the trading chain by the agent after ENS approval; a cross-chain read of the ENS record would remove that step.
- Mainnet deployment, audit, fee optimisation (listed as open in [1]), and multi-pair rebalancing are out of scope.

Bibliography

- [1] S. Ko, “Partially Active Automated Market Makers,” *arXiv preprint arXiv:2602.09887*, 2026.
- [2] H. Kim and Y.-S. Park, “Bounding LVR in AMMs via Secant-Tangent Divergence and Collateralized Liquidity Scaling,” *arXiv preprint arXiv:2605.19267*, 2026.

10 Appendix: sponsor requirements to file and line

Requirement	Where
linch: official Aqua/SwapVM contracts, router redeployment	contracts/src/aqua/TideRouter.sol:21, submodule contracts/lib/swap-vm (main), Aqua registry 0x1111113C...a90a
linch: custom opcodes, slots, order	contracts/src/aqua/ instructions/TideProgram.sol:24-32 ActiveSplit.sol:59, VirtualXYCSwap.sol:48, BufferGuard.sol:54, TideOpcodes.sol:16
linch: onchain transfers in demo	Sepolia tx 0xcaealc64...457773; contracts/script/fork-demo.sh
Fee-rebate bound (Prop. 4)	contracts/src/lib/TideMath.sol:115, contracts/src/TideParams.sol:43,67, tests contracts/test/aqua/TideAqua.t.sol:291,318
linch: commit history	small commits from Sep 25 evening, no squash
Uniswap: hook + lines	contracts/src/ v4/TideHook.sol:109,146,151,205,211
Uniswap: FEEDBACK.md, form	FEEDBACK.md (root)
ENS: Sepolia ENSv2, central, no hard-coded values	client/scripts/ens-setup.ts, client/src/lib/ens/client.ts:52,75,80, records on eth-usdc.tide.eth
ENS: live demo, open source	dashboard client/, records readable via Universal Resolver
World: dev environment, full journey, denied path, backend validation	client/src/lib/world.ts:78,109, client/src/lib/agent.ts:98,146, client/scripts/test-agent-flow.ts
World: debrief	docs/world-debrief.md
Curvegrid: README summary, team, setup and tests	README.md